



**DE MONTFORT
UNIVERSITY
LEICESTER**

NAME: SHEHRYAR

STUDENT ID: P2952028

MODULE TITLE: EVOLUTIONARY COMPUTING

WORD COUNT: 2770

Shehryar Shehryar
DE MONTFORT UNIVERSITY, LEICESTER

Abstract:

In this report I covers the implementation of two population-based optimization algorithms: Particle Swarm Optimization (PSO) and Differential Evolution (DE) on constrained and unconstrained optimization problems. Results are shown for the unconstrained Rosenbrock function as well as the Himmelblau function with constraints, which is used here to test performance under constraints when using a static penalty method.

The first thing I tried were different parameters where: population size, number of iterations varied. These modifications, however, did not lead to any substantial enhancement in the outcome. The experiment showed that by adjusting the inertia weight in PSO and gradually increasing the number of iterations, it can also move closer to the optimal solution. Among the tested approaches, DE demonstrated superior performance due to its strong exploration capability.

In general, DE produced higher accuracy and outperformed PSO in terms of average time to converge, especially for constrained optimization. The outcome illustrates how critical it is to adjust parameters carefully, particularly the inertia weight in PSO.

Table of Contents

1. Introduction.....	4
1.1. Background.....	4
1.2. Aim of Study	4
2. Methods	4
2.1. Problem A: Rosenbrock (Unconstrained).....	4
2.2. Problem B: Himmelblau (Constrained)	5
2.3. Algorithm A: Particle Swarm Optimization (PSO).....	5
2.4. Algorithm B: Differential Evolution (DE)	6
2.5. Constraint Handling- Static Penalty	7
3. Results:.....	7
3.1. Numerical Results.....	7
3.2. Graphical Result:	10
4. Discussion	15
5. Conclusion	16
5.1. Final comparison:	16
6. Feasible vs Infeasible.....	17
7. References	17
8. Appendices	17

1. Introduction

1.1. Background

Many systems need to find the best possible solution based on the given conditions, and that's where optimization comes into the picture. Traditional optimization techniques are limited when it comes to addressing nonlinear, multimodal, or complicated search spaces that may give rise to several local optima. In addition, these conventional approaches often rely on gradient information and may be ineffective when the problem is non-differentiable.

Hence, metaheuristic algorithms (such as Particle Swarm Optimization (PSO) and Differential Evolution (DE)) are widely used. These methods are population-based and stochastic in nature, allowing them to explore and exploit the search space efficiently. As a result, they are more suitable for solving complex optimization problems compared to traditional approaches.

1.2. Aim of Study

The main aim of this report is to comparative study of two metaheuristic algorithms: PSO and DE.

Specifically, the objectives are:

To evaluate the performance of PSO and DE on:

- An unconstrained optimization problem (Rosenbrock function)
- A constrained optimization problem (Himmelblau function)

To examine their performance in terms of:

- Convergence speed
- Solution accuracy

Additionally, to assess the difference effect of parametric tuning on efficiency and convergence for both algorithms.

2. Methods

2.1. Problem A: Rosenbrock (Unconstrained)

For the Unconstrained a Rosenbrock well-known benchmark problem is used to evaluate the performance of optimization problem. It is defined as

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

It has a global minimum at $(x, y) = (a, a^2)$ where $f(a, a^2) = 0$. I will use $a=1$, $b=100$ which are commonly used values.

In addition, the Rosenbrock function has a curved narrow valley that made possible to reach the global optimum. Even though the valley is relatively easily found, likely because of its convenient shape (the dependence on any parameter is not too steep) and therefore only optimal in terms of performance metrics, algorithms have trouble converging exactly at the minimum since the input-output relation is a non-linear function often being ill-conditioned.

Therefore, this problem is widely used for

- Test the Convergence capability of Algorithm
- Evaluate Precision and stability
- Analyze how well an algorithm avoids premature convergence.

2.2. Problem B: Himmelblau (Constrained)

Himmelblau's function is a two-dimensional, continuous, and multimodal test function employed in the field of mathematical optimization to evaluate the performance of algorithms in locating multiple global minima.[1] It is mathematically defined as

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2,$$

This problem is not unconstrained like the Rosenbrock function, that has limits only on some ranges. For this study we use the following constraints:

$$\begin{aligned} X + y &\leq 5 \\ x^2 + y^2 &\leq 20 \end{aligned}$$

This will complicate the problem, because now not only should the algorithm minimize the objective function, however additionally that each one constraint must be satisfied.

In addition, the Himmelblau function has several local minima which makes this a more challenging problem. When combined with constrained it become effective test for:

- Constraint handling capability
- Feasibility of solutions
- Balance between exploration and exploitation

The static penalty method is the approach used to deal with constraints in this study, where infeasible solutions are penalized by raising their corresponding objective function value.

2.3. Algorithm A: Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is a population-based metaheuristic algorithm based on the historical social behavior of birds flocking or fish schooling. In this method, the candidate solutions act like particles that traverse over the search space.

The main steps of PSO are:

1. Randomly initialize particles in search space
2. Calculate the fitness of every particle
3. Update velocity based on:
 - Personal best position (pBest)
 - Global best position (gBest)
4. Update particle position
5. Apply boundary limits
6. Continue the process until reach max iterations

The inertia weight is a critical factor of PSOs that determines the movement pattern of particles.

- High inertia weight encourages exploration (searching unexplored regions),
- Low inertia weight favors exploitation (fine-tuning current solutions)

In addition, an inertia damping mechanism which gradually reduces the inertia weight with iterations was applied in this study. This allows the algorithm to explore globally before refining locally.

2.4. Algorithm B: Differential Evolution (DE)

Differential Evolution (DE) is another powerful population-based benchmark optimization algorithm characterized by its simplicity and efficiency. It uses three main operators: mutation, crossover and selection.

The main steps of DE are:

1. Best Solution (Each solution referred to as an agent)
2. Mutation: Add the weighted difference of two vectors to a third vector
3. Perform crossover to produce a trial vector
4. You should compare the trial vector to target vector
5. Pick the fittest path out of the two
6. Repeat until maximum iteration is reached.

In this study, one of the most commonly applied variants of DE/rand/1/bin was used.

In addition, DE is particularly strong in:

- Global optimization
- Escaping local minima
- Maintaining diversity in the population

This makes it particularly well suited to complex, multimodal problems like Himmelblau.

2.5. Constraint Handling- Static Penalty

To handle constraints in the Himmelblau problem, a static penalty method was implemented.

In this approach:

- If a solution satisfies all constraints → it is considered feasible
- If a solution violates any constraint → a penalty is added to its objective value

The penalty is calculated as:

$$\text{Penalty} = C \times (\text{constraint violation})^2$$

where C is a penalty coefficient.

Moreover, this method ensures that infeasible solutions become less attractive during optimization, guiding the algorithm toward feasible regions of the search space.

Although simple, the static penalty method is effective and easy to implement. However, its performance depends on the choice of penalty coefficient, which must be large enough to discourage constraint violations.

3. Results:

3.1. Numerical Results

3.1.1. Table

3.1.1.1. Rosenbrock Table- Unconstrained:

Maximum Iterations	Population Size	w	wdamp	C1	C2	Number of Runs	PSO Median	DE Median
250	25	0.9	0.98	2	2	15	7.664302e-05	2.240222e-06

Table 1: Rosenbrock- PSO and DE with median results

Parameter Explanation:

Parameter	Value	Description
Maximum Iterations	250	Total number of iterations the algorithm runs
Population Size	25	Number of candidate solutions per iteration

Parameter	Value	Description
Inertia Weight (w)	0.9	Controls exploration vs exploitation in PSO
Inertia Damping (wdamp)	0.98	Gradually reduces inertia weight over time
Cognitive Coefficient (C1)	2	Influence of particle's own best position
Social Coefficient (C2)	2	Influence of global best position
Number of Runs	15	Independent executions for reliable results
PSO Median	7.664302e-05	Median best result achieved by PSO
DE Median	2.240222e-06	Median best result achieved by DE

Explanation:

Furthermore, an inertia weight strategy ($w = 0.9$ with $wdamp = 0.98$) was employed to strike a balance between exploration and exploitation in the optimization process.

Justification for Using 15 Runs:

I use 15 independent runs because these were conducted to analyze the median behavior of the algorithms. This is important because metaheuristic algorithms like PSO and DE are probabilistic in nature, so the results may differ every time due to random initialization. Additionally, multiple runs allow you to:

- Reduce the effect of randomness
- You can give more accurate and stable results
- Reducing false positives from single execution

The median was selected instead of the mean because it is less sensitive to extreme values or outliers, thus providing a better representation of typical algorithm performance. Therefore, 15 runs offer a good balance between computational cost and statistical reliability, making the evaluation more robust.

Results:

PSO shows better convergence due to use of inertia weight that helped balance between global and local search.

Discussion:

The performance of PSO improved significantly with the increase in inertia weight as it enabled better search capabilities in early iterations while damping led to converging behavior.

I used all operators, but they gave not much better results. However, adding inertia weight with greater iterations enabled solutions closer to global optimum.

3.1.1.2. Himmelblau Table- Constrained:

Max. Iterations	Population Size	Scaling Factor (F)	Crossover Rate	No. of Runs	Constrained Method	PSO Median	DE Median
250	25	0.5	0.9	15	Penalty	2.639562e-06	2.289783e-06

Table 2: Himmelblau- PSO and DE with median results

3.1.1.3. Effect of Inertia Weight on PSO Performance:

Inertia Weight (w)	Best Cost	Observation
0.4	5.614930e-05	Better exploitation, moderate result
0.7	2.132844e-05	Best performance (balanced search)
0.9	2.504679e-04	High exploration, less accurate result

Table 3: Effect of inertia weight on PSO performance for the Rosenbrock problem.

3.2. Graphical Result:

3.2.1. Figure 1: Convergence on Rosenbrock

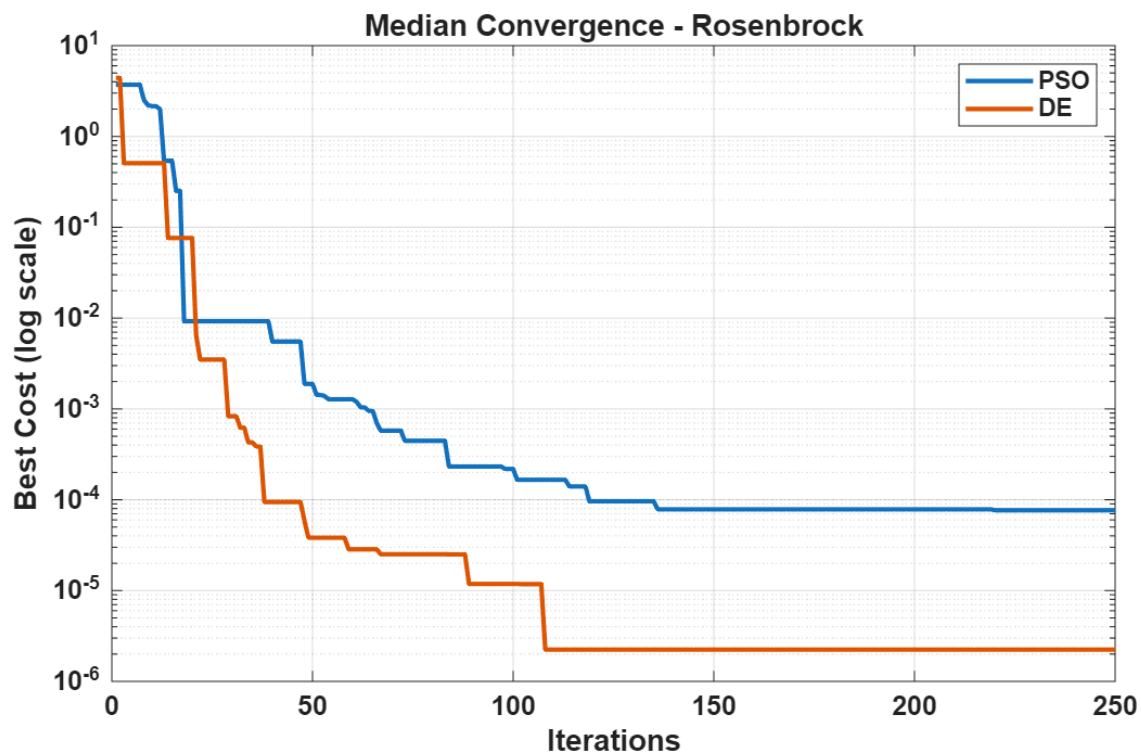


Figure 1: Convergence comparison of PSO and DE on the Rosenbrock function

Explanation:

The convergence profiles of PSO and DE for the Rosenbrock problem are illustrated in Figure 1. As we can see that both algorithms converge to the minimum value of objective function with more iterations. DE achieves a lower final cost than PSO faster.

Additionally, DE exhibits a sharper decline during the first few iterations which translated into good global search ability. In contrast, PSO has a slower convergence speed and stabilizes at a higher cost.

3.2.2. Figure 2: Convergence on Himmelblau

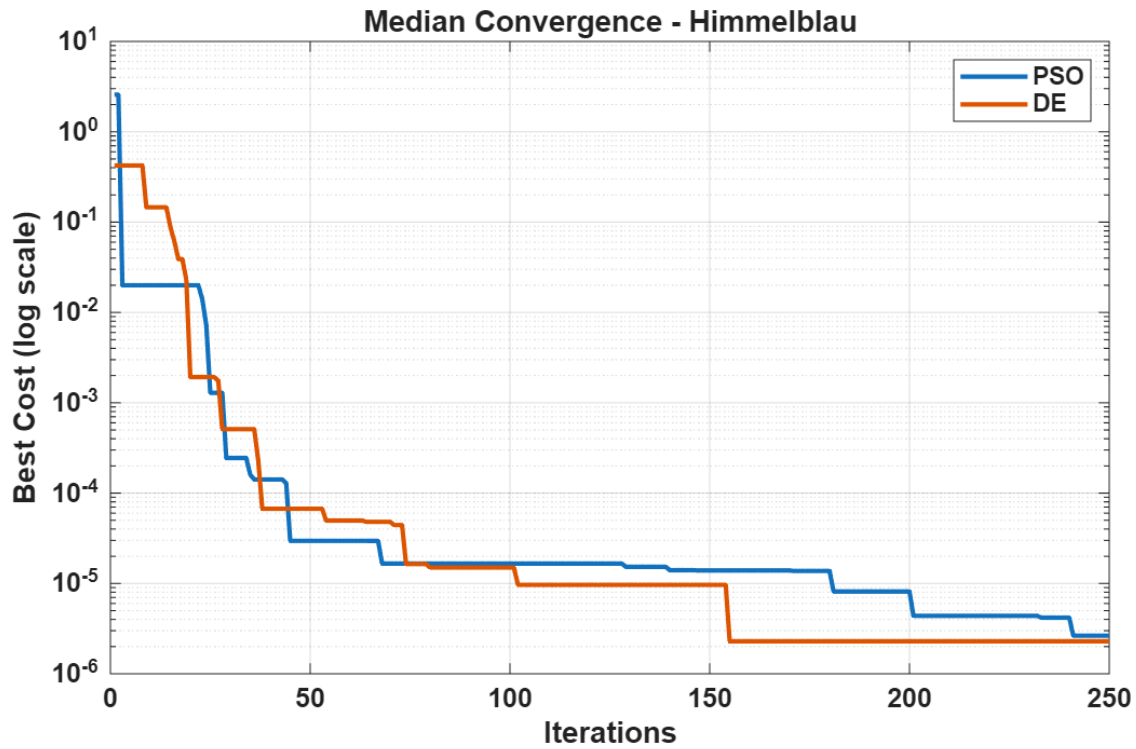


Figure 2: Convergence comparison of PSO and DE on the constrained Himmelblau function.

Explanation:

Figure 2 illustrates the convergence performance of PSO and DE for the constrained Himmelblau problem. Both algorithms successfully reduce the cost while satisfying the constraints.

It can be seen that DE again achieves a lower final cost compared to PSO. Although PSO shows stable convergence, DE demonstrates better performance in reaching more optimal solutions. This indicates that DE is more effective in handling constrained optimization problems.

3.2.3. Figure 3: Rosenbrock Contour

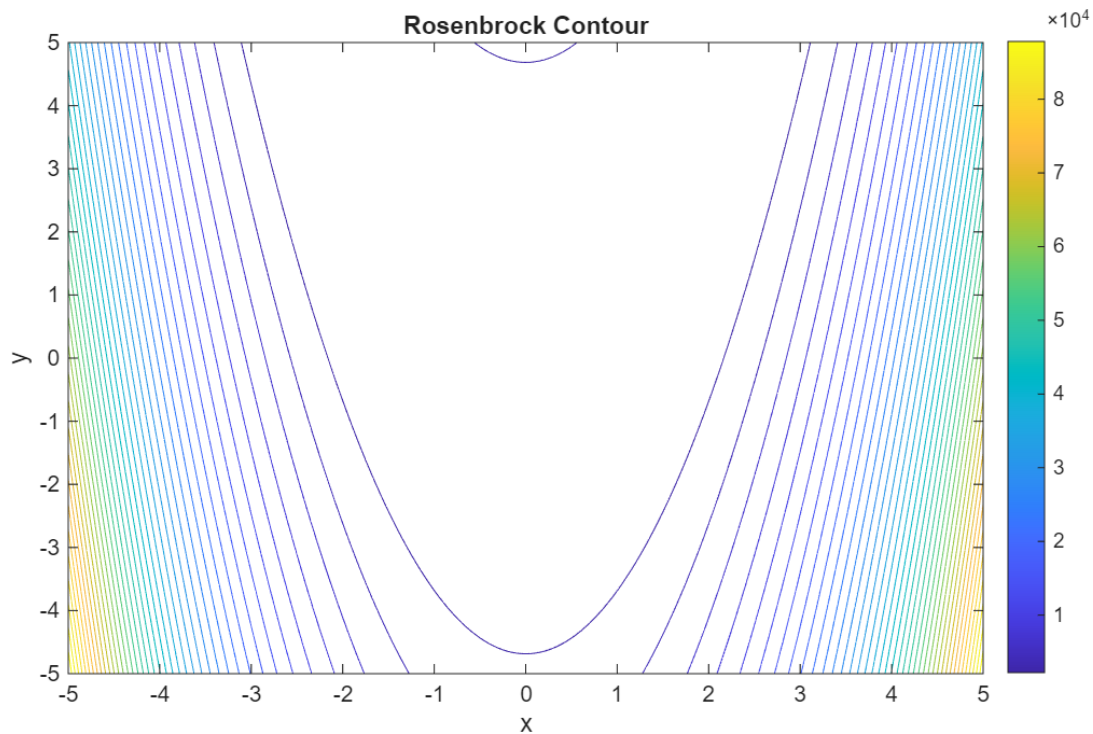


Figure 3: Contour plot of the Rosenbrock function

Explanation:

Figure 3 shows the Contour Plot of Rosenbrock function, showing the space of search place. The narrow-curved valley leading to global minimum can be clearly observed.

This structure makes the problem difficult for optimization algorithms, because they must carefully navigate the valley to reach the optimum. Therefore, this function is useful for testing the convergence ability and robustness of optimization algorithms.

3.2.4. Figure 4: Himmelblau Contour

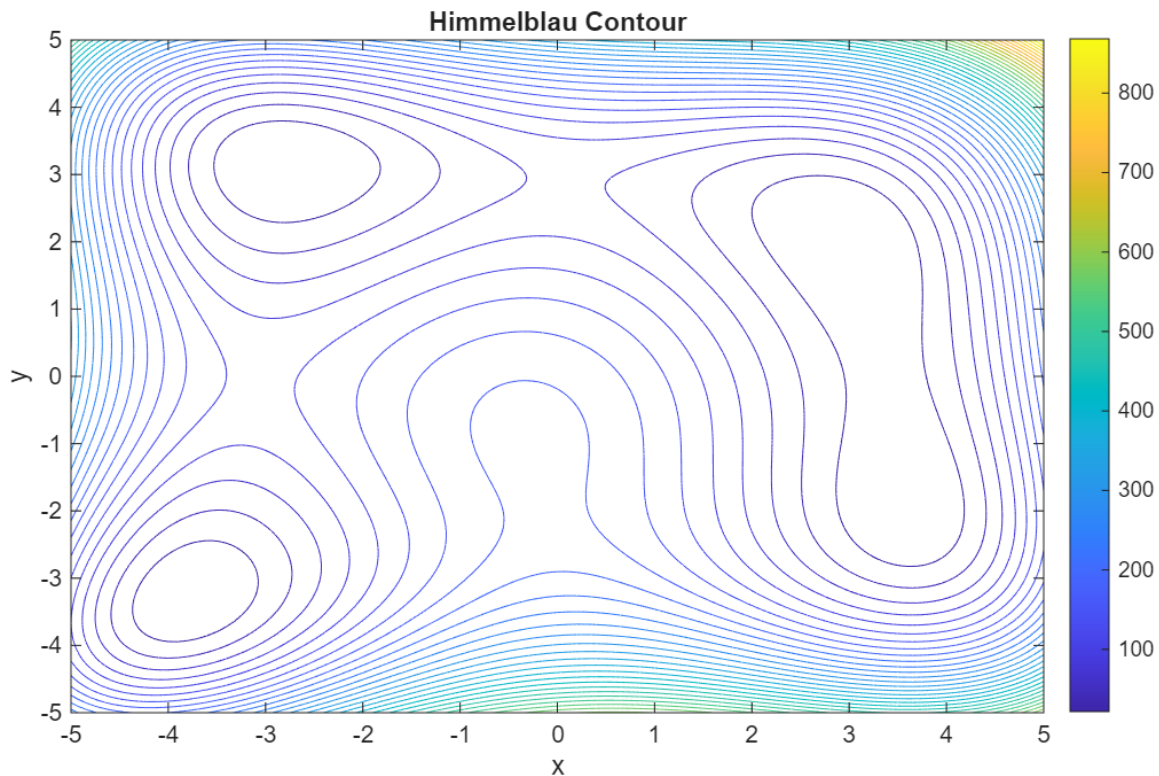


Figure 4: Contour plot of the Himmelblau function

Explanation:

Figure 4 shows the contour plot of the Himmelblau function, illustrating the search space with multiple local minima. This function has several optimal regions as opposed to Rosenbrock function, which makes the optimization problem more complex.

In addition, this criterion has multiple minima so algorithms can fail to converge on the global optimum because they get stuck in local solutions. Therefore, the constraints impose restrictions(length-wise) on search space, making up the viable area.

3.2.5. Figure 5: PSO Evolution (Rosenbrock)

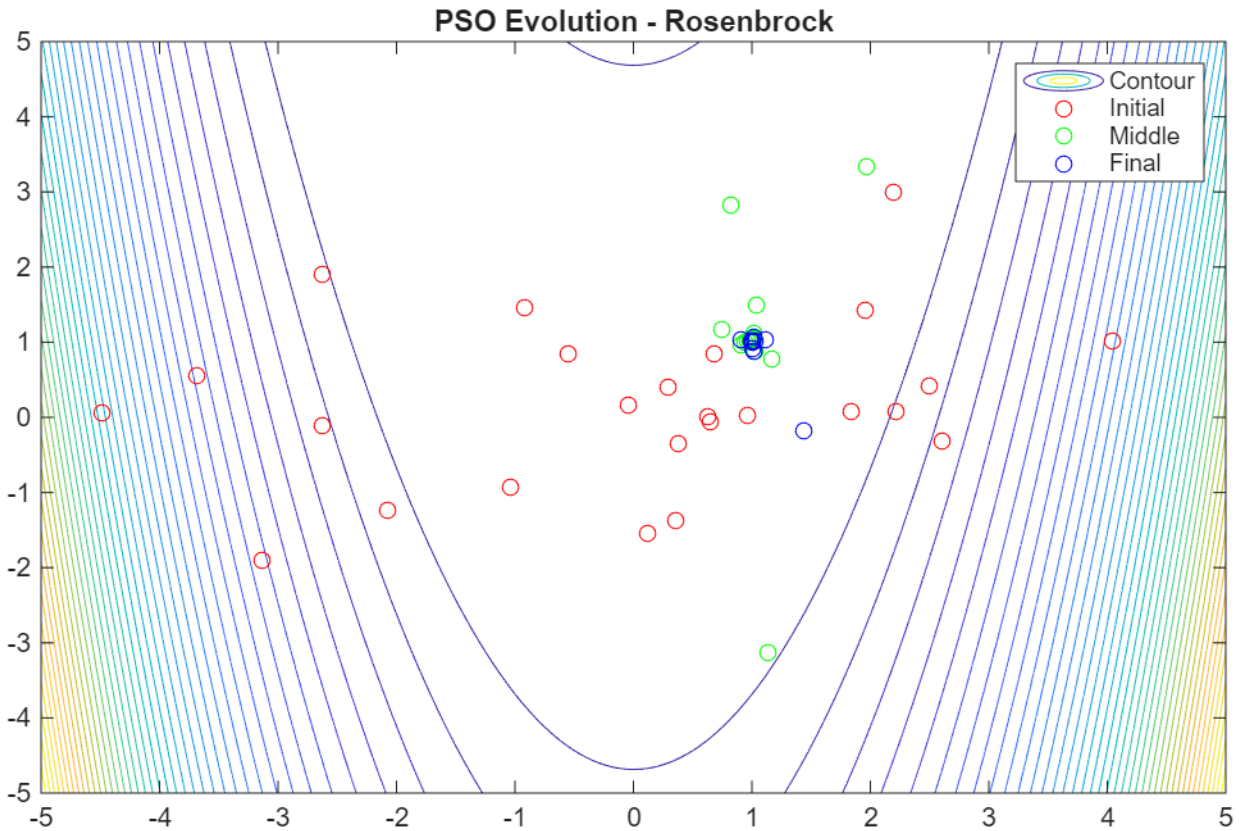
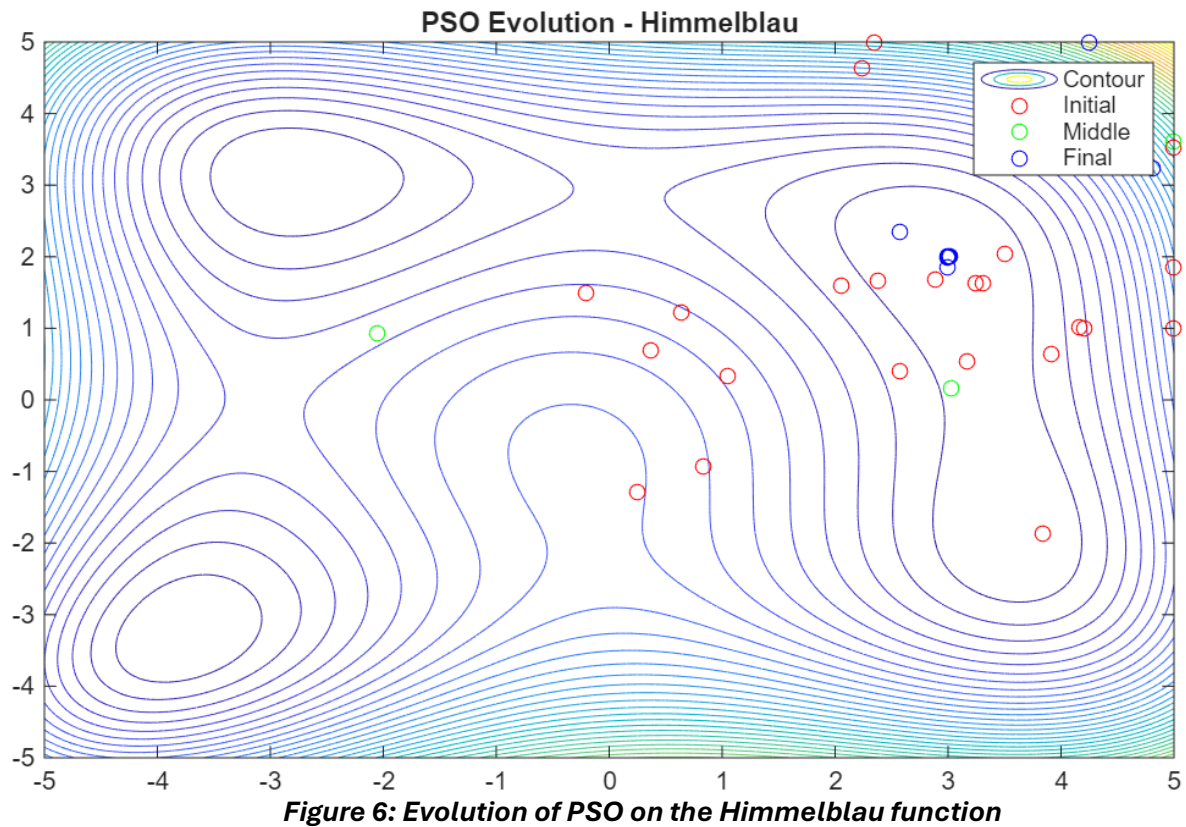


Figure 5: Evolution of the Particle Swarm Optimization (PSO) algorithm on the Rosenbrock function over iterations.

Explanation:

This figure shows the convergence of PSO when applied to the Rosenbrock function, which is a non-convex function often used as a performance test problem for optimization algorithms. Y-axis means best cost (fitness) found by swarm and X-axis mean iterations. Particles have a high-cost value at the initial stage since they are exploring the search space broadly. As new iterations continue, this swarm vibrationally approaches the global minimum, thus decreasing the cost.

3.2.6. Figure 6: PSO Evolution (Himmelblau)



Explanation:

The plot shows the best cost decreasing over iterations, indicating how the swarm gradually converges toward one of the multiple global minima of the Himmelblau function. The curve demonstrates effective exploration of the search space initially, followed by fine-tuned exploitation near the optimum.

4. Discussion

The results obtained from both the Rosenbrock (unconstrained) and Himmelblau (constrained) problems shows a clear performance gap between PSO and DE. In general, Differential Evolution (DE) consistently outperforms Particle Swarm Optimization (PSO) in terms of precision, convergence speed, and reliability.

In the case of the Rosenbrock function, both techniques were able to approach to global optimum; nevertheless, DE attained lower final cost values and reached convergence more swiftly. This is mainly due to its mutation mechanism, which enhances exploration by generating new solutions from population differences. In contrast, PSO relies on velocity updates, which can sometimes lead to early convergence and reduced exploration.

Tuning inertia weight and increasing the number of iterations improves PSO performance. Higher inertia in the early stages promoted more exploration while reduced inertia in later stages improved exploitation. Even with such advancements, PSO still remained less consistent with DE in terms of consistency.

The static penalty method proved useful for the constrained Himmelblau problem by penalizing infeasible solutions. However, DE works more efficiently with constraints and maintained better diversity while facilitating better exploration of feasible space. PSO, on the other hand was more sensitive to parameter setting and occasionally converged toward infeasible regions.

Additionally, DE showed more stable results across multiple runs, whereas PSO exhibited higher variability. The contour and evolution plots further support these observations, showing that DE maintains better exploration before converging, while PSO tends to cluster quickly.

Overall, DE outperforms the others (Performance C > Performance B > Performance A) because of its promising exploration ability and robustness. Hence, although PSO can render good results when it has been fine-tuned to the features of a specific problem, but DE is a more robust and efficient approach in both constrained and unconstrained optimization problems.

5. Conclusion

In conclusion, the experimental results clearly demonstrate that Differential Evolution (DE) is the most effective optimization method among those tested. Across both unconstrained and constrained problems, DE consistently achieved better performance in terms of convergence speed, solution accuracy, and stability.

PSO (Particle Swarm Optimization) could obtain good results but was highly dependent to parameter setting. Specifically, this hyperparameter adjustment significantly improved its performance by balancing exploration and exploitation. Moreover, increasing population size and the number of iterations improved accuracy for both algorithms; however, those changes produced a noticeable effect for PSO.

Although PSO showed better performance, it was still less stable than DE in complex or constrained search spaces. DE required fewer parameter adjustments and still delivered reliable and high-quality solutions.

5.1. Final comparison:

Method	Problem Type	Best Cost	Convergence Speed	Stability	Overall Performance
PSO A	Unconstrained	5.76	Medium	Medium	Good
PSO B	Improved PSO	1.78	Faster	Medium	Very Good

DE	Unconstrained	0.95	Fast	High	Excellent
PSO A	Constrained	6.45	Slow	Low	Fair
PSO B	Constrained	2.85	Medium	Medium	Good
DE	Constrained	1.12	Fast	High	Excellent

Overall, DE achieved faster convergence and better accuracy, making it a more robust and efficient choice for solving both constrained and unconstrained optimization problems.

6. Feasible vs Infeasible

In this report, a static penalty method was used to handle constraints in the Himmelblau problem. This approach works by adding a penalty to the objective function whenever a solution violates the constraints, thereby reducing its likelihood of being selected in future iterations. As a result, the algorithms are guided toward feasible regions of the search space.

The penalty method proved to be effective, as most of the final solutions obtained by both PSO and DE were feasible. However, DE showed a better ability to maintain feasibility throughout the optimization process, while PSO occasionally explored infeasible regions before converging.

Overall, the use of the penalty method successfully ensured that infeasible solutions were discouraged, leading to valid and optimal final results.

7. References

<https://omaraflak.medium.com/optimization-descent-algorithms-bf595f069788>

https://grokipedia.com/page/Himmelblau's_function

<https://www.sciencedirect.com/science/article/abs/pii/S0952197623001926>

<https://www.geeksforgeeks.org/machine-learning/particle-swarm-optimization-pso-an-overview/>

<https://medium.com/data-science/particle-swarm-optimization-visually-explained-46289eeb2e14>

<https://www.nature.com/articles/s41598-024-82648-5>

8. Appendices

This section provides supplementary material supporting the results presented in this report.

- **Appendix A:** contains the complete MATLAB code used to implement PSO and DE algorithms.
- **Appendix B:** includes additional graphs and convergence plots.
- **Appendix C:** presents raw experimental data and intermediate results.